

Deep Adaptive Design of Experiments for Dynamical Systems

Arno Strouwen · PumasAI · KU Leuven

Sebastian Micluța-Câmpeanu · JuliaHub · University of Bucharest

JuliaCon 2026

Notation and goal

- θ : the unknown parameters.
- u : the design.
- y : the observations.

State of belief *before* the experiment:

$p(\theta)$ the **prior**

After the experiment, Bayes' rule updates it:

$$p(\theta \mid y, u) = \frac{p(y \mid \theta, u) p(\theta)}{\int p(y \mid \theta', u) p(\theta') d\theta'} \quad \text{the **posterior**}$$

Bayesian experimental design: choose u to make the posterior as sharp as possible.

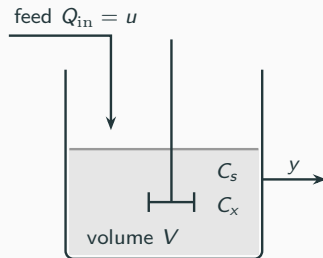
Concretely: a fed-batch bioreactor

$$\dot{C}_s = -\frac{\mu(C_s)}{Y_{xs}} C_x + \frac{Q_{in}}{V} (C_{s,in} - C_s),$$

$$\dot{C}_x = \mu(C_s) C_x - \frac{Q_{in}}{V} C_x,$$

$$\dot{V} = Q_{in}, \quad \mu(C_s) = \frac{\mu_{max} C_s}{K_s + C_s}.$$

Substrate is pumped in, biomass eats it and grows, and the vessel fills up.



- **Design** $u = Q_{in}$: the feed rate, which we may change once an hour.
- **Observations** $y = C_s + \text{noise}$: the substrate concentration, sampled once an hour.
- **Unknown** $\theta = (\mu_{max}, K_s)$: the two constants in the growth law μ .

Which feed profile makes the posterior over μ_{max} and K_s as sharp as possible?

Scoring a design

How sharp is a posterior? Measure it by its **entropy**, small when sharp and large when vague:

$$H[p(\theta \mid y, u)] = - \int p(\theta \mid y, u) \log p(\theta \mid y, u) d\theta.$$

y is not measured yet, so we average over every observation the design could produce:

$$u^* = \arg \min_u \mathbb{E}_{y|u} [H[p(\theta \mid y, u)]].$$

Scoring a design means averaging over the data it might produce.

In our examples only part of θ is of interest: split $\theta = (\theta_T, \theta_N)$ into the kinetics θ_T and nuisance θ_N (sensor noise, initial biomass), and score only the marginal posterior of the target,

$$u^* = \arg \min_u \mathbb{E}_{y|u} [H[p(\theta_T \mid y, u)]], \quad p(\theta_T \mid y, u) = \int p(\theta \mid y, u) d\theta_N.$$

We will not go into what that does to the estimator here; it is in the paper: *Deep Adaptive Model-Based Design of Experiments*, arXiv:2603.16146.

Why that is expensive

Up to a constant, the entropy criterion can be rewritten as

$$\max_u \mathbb{E}_{\theta, y|u} [\log p(y | \theta, u) - \log p(y | u)].$$

The first term is easy: simulate the model and evaluate its likelihood.

The second term is the **evidence**, an integral over every parameter the prior allows:

$$p(y | u) = \int p(y | \theta, u) p(\theta) d\theta.$$

Replace it by a sample average and you get a loop inside a loop:

$$\frac{1}{N} \sum_{n=1}^N \left[\log p(y_n | \theta^{(n)}, u) - \underbrace{\log \frac{1}{L} \sum_{\ell=1}^L p(y_n | \theta^{(\ell)}, u)}_{\text{inner loop}} \right]$$

Every one of those likelihoods is an ODE solve.

$N \times L$ ODE solves to score *one* candidate design, and the optimizer wants many.

From static schedules to policies

Static: choose the whole input sequence $u_{1:K} = (u_1, \dots, u_K)$ before the experiment.

Adaptive: choose u_k only after seeing the history h_{k-1} of past inputs and measurements. The optimum alternates a decision and an expectation at every step:

$$V = \min_{u_1} \mathbb{E}_{y_1} \min_{u_2} \mathbb{E}_{y_2} \cdots \min_{u_K} \mathbb{E}_{y_K} [H[p(\theta \mid h_K)]] .$$

The usual workaround: after each measurement, update the posterior and re-optimize only the next input — inference and optimization, between two measurements.

Instead: let a neural network be the rule, $u_k = \pi_\phi(h_{k-1})$, and optimize its weights ϕ by policy gradient. The nesting collapses into one expectation over simulated episodes, and the best policy is as good as V .

Trained once, offline; in the lab, each step is one forward pass.

What one gradient step costs

One decent estimate of the criterion (a contrastive bound, sPCE) takes $L + M + 2$ ODE solves per episode — one rollout, $L = 464$ prior decoys, $M = 466$ nuisance draws — over a batch of $B = 53,648$ episodes:

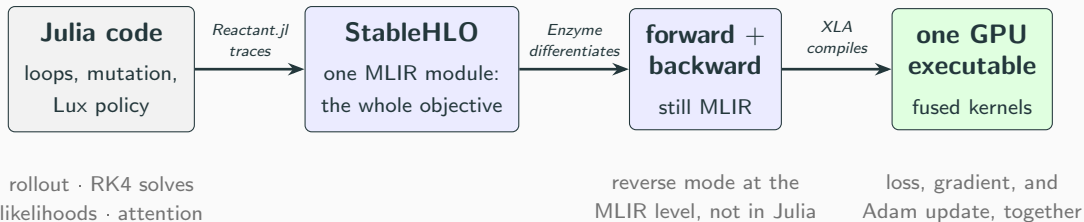
$$\underbrace{932}_{L+M+2} \times \underbrace{53,648}_B \approx 5 \times 10^7 \text{ trajectories} \quad \approx 3.5 \times 10^{10} \text{ RK4 steps.}$$

And we need its gradient, not its value — a thousand times, once per optimizer step.

That is a lot of ODE solves: 15.4 s per gradient step, 4.3 hours on one H100.

So the batch has to live on a GPU, the whole objective has to become one compiled program, and the solver has to be differentiated for us.

The stack: from Julia code to one GPU executable



Lux.jl: weights are explicit arguments, so the loss is a pure function — what a tracing compiler needs.

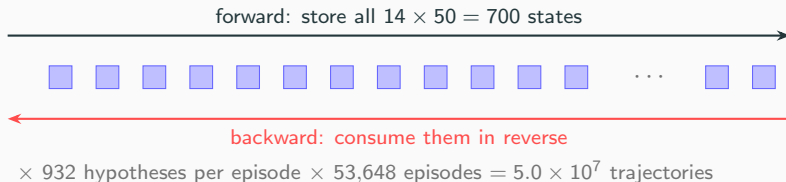
SimpleDiffEq: its RK4 step! drops into the traced loop unchanged.

Optimisers.jl: the update rides along via `Lux.Training.single_train_step!`.

One gradient step becomes one compiled executable: no Julia left in the inner loop.

The catch: reverse mode must remember the forward pass

To run the chain rule backwards, Enzyme needs the state at *every* RK4 substep of every trajectory — visited in reverse, after the forward pass is done.



$$\underbrace{3.5 \times 10^{10}}_{\text{stored states}} \times \underbrace{3}_{(C_s, C_x, V)} \times \underbrace{4 \text{ bytes}}_{\text{Float32}} \approx 420 \text{ GB} \quad \text{— and that is the state alone.}$$

tape, all at once $\approx 420 \text{ GB}$

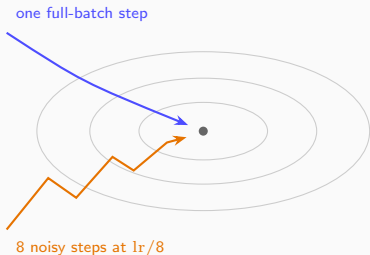
one H100 80 GB

First cut: the gradient is an expectation

The objective is an expectation over simulated episodes; a sample average of gradients is **unbiased at any B** — batch size buys *variance*, not correctness:

$$\nabla_{\phi} \mathcal{L} = \mathbb{E}_{\theta \sim p(\theta), \varepsilon} [\nabla_{\phi} \ell(\phi; \theta, \varepsilon)] \approx \frac{1}{B} \sum_{b=1}^B \nabla_{\phi} \ell(\phi; \theta^{(b)}, \varepsilon^{(b)}).$$

Stochastic gradient descent, then: 8 strips of 6,706 episodes, one Adam step per strip at $lr/8$ — an eighth of the activations alive at once.

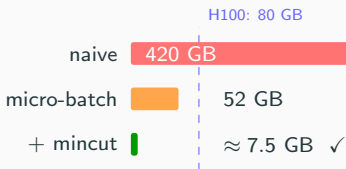


```
for _k in 1:8    # micro-batches; Adam at lr/8
    data = prepare_batch(rng, ...) # fresh draws
    _, loss, _, ts = single_train_step!(
        AutoEnzyme(), spce_loss, data, ts)
end
```

And there is no dataset to shortchange: every strip is freshly simulated from the prior.

Second cut: checkpoint the substeps

A strip still stores all 50 RK4 states per control step. Enzyme's min-cut keeps $\approx\sqrt{N}$ of them; the backward pass recomputes the rest from the nearest checkpoint — twice the forward FLOPs, a seventh of the memory.



In the code, the entire plan is one annotation:

```
@trace mincut=true for _ in 1:n_substeps
    u = rk4_step(u, θ, d, dt_sub)
end
```

No manual checkpoints, no custom backward pass — memory becomes a tracing annotation.

The policy: a small transformer over the history

```
const policy = @compact(  
  input_proj = Dense(2 => 32),  
  rms1 = RMSNorm((32,)), mha = MultiHeadAttention(32; nheads = 4),  
  rms2 = RMSNorm((32,)), ff = Chain(Dense(32 => 64, gelu), Dense(64 => 32)),  
  output_head = Dense(32 => 1; init_bias = (rng, dims...) -> fill(-4.0f0, dims...)),  
) do x  
  # x: (2, K, B) -- the (y_k, u_k) pairs so far  
  x = input_proj(x)  
  x = x .+ reshape(sinusoidal_pe(size(x, 2)), 32, :, 1) # make time explicit  
  attn, _ = mha(rms1(x))  
  x = x + attn  
  x = x + ff(rms2(x))  
  @return ACTION_HI .* sigmoid.(output_head(x[:, end, :])) # next feed rate in [0, 10]  
end
```



The history is a fixed 14-slot buffer, zero where nothing has been measured yet, so the future cannot leak in. The positional encoding is where we deviate from DAD: for a dynamical system, hour 3 and hour 11 are not interchangeable. 8.6k parameters: next to the ODEs, free.

The objective, as written — and the same loop in JAX

```
for step in 1:N_STEPS                                # roll out under the current policy
    action, st = model(input_buffer, ps, st)
    u = integrate(u,  $\theta_{\text{dyn\_true}}$ , action, DT, N_SUBSTEPS)
    y = observe_noisy(u,  $\theta_{\text{obs\_true}}$ ,  $\epsilon$ , step)      #  $y = g(x) + \sigma \cdot \epsilon$ , with  $\epsilon$  drawn up front
    input_buffer[1, step, :] .= y                     # mutation is fine: it gets traced away
end
```

Julia + Reactant

```
function integrate(u,  $\theta$ , d, dt, n_sub)
    @trace mincut=true for _ in 1:n_sub
        u = rk4_step(u,  $\theta$ , d, dt / n_sub)
    end
    return u
end
```

Python + JAX

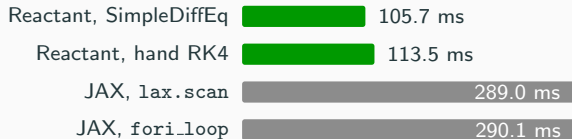
```
@jax.checkpoint
def integrate_scan(u, theta, Q_in):
    dt_sub = DT / N_SUBSTEPS
    def body(u, _):
        return rk4_step(u, theta, Q_in, dt_sub), None
    u, _ = lax.scan(body, u, None, length=N_SUBSTEPS)
    return u
```

- Both emit StableHLO for XLA: identical arithmetic, different front end. Hypotheses are one more array axis — u is $(3, L+1, B)$ — hand-batched on both sides, vmap or not.
- Reverse-mode memory: `jax.checkpoint` placed by hand, versus `mincut=true` letting Enzyme choose the cut.

Reactant versus JAX at a matched workload

Same estimator, same shapes: $L = 255$, $M = 256$, $B = 512$, Float32, 14 steps of 50 RK4 substeps — 262,656 trajectories per timed step, on one GPU.

loss + gradient, per step (median)



first call (compile)



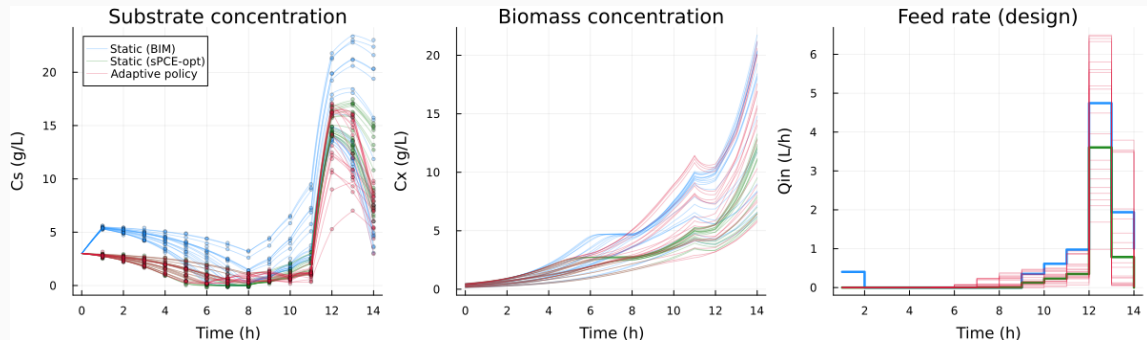
Per step: Reactant is $\sim 2.7\times$ faster — its `@trace` for lowers to a tighter XLA while than `scan/fori`.

First call: JAX compiles $\sim 70\times$ faster. Iterating on the model, you feel it; amortized over a training run, you don't.

SimpleDiffEq $\approx$ hand RK4: same loss to 5 digits; the library integrator really is a free drop-in.

Loss + gradient only, data on device, sync before stopping; median of 20 after warmup. RTX 4080 SUPER, Reactant 0.2.273, JAX 0.10.2 — a statement about this program, not the two languages.

What the learned policy does

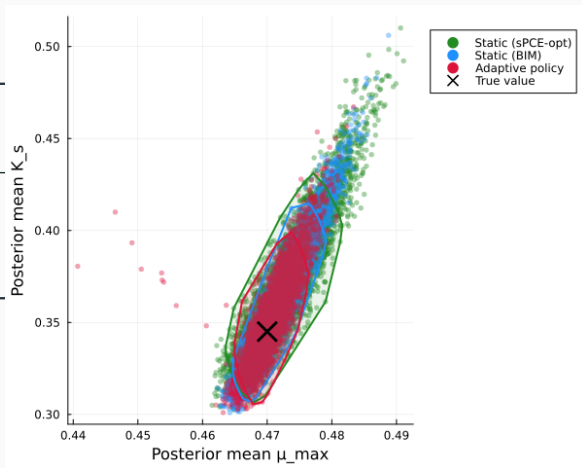


Twenty draws of the true ($\mu_{\max}$, K_s), the same twenty for each strategy.

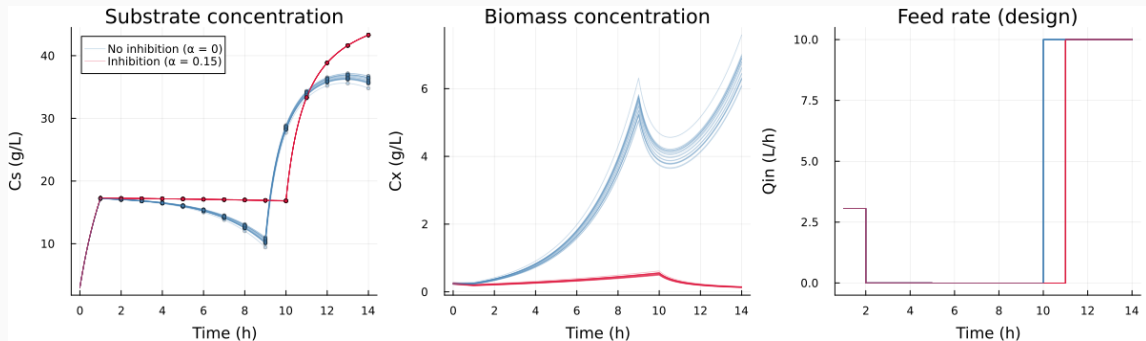
Sharper posteriors

Design	sPCE (nats)	RMSE ($\times 10^3$)	
		$\mu_{\max}$	K_s
Adaptive policy	3.68	3.1	24.9
Static (BIM)	3.45	4.4	34.3
Static (sPCE-opt)	3.34	5.6	41.5

Same number of measurements,
about 30% lower error.



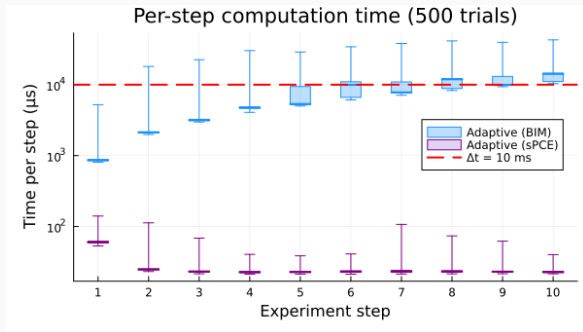
Two runs, two different experiments



Kinetics fixed; two values of the inhibition strength, twenty nuisance draws each.

The design adapts to the true parameters.

Real time: a DC motor on a 10 ms clock



Adaptive (BIM) — re-optimized online. After every measurement: update the posterior, optimize the next voltage. All of it inside the 10 ms window.

Adaptive (sPCE) — the amortized policy. All of that happened offline, before the motor was switched on. Online, one forward pass.

Amortization is what makes adaptive design real-time.

Where this meets SciML

SimpleDiffEq + Reactant. We can use SimpleDiffEq via the low-level `init/step!` interface, needed for the `mincut` annotation:

```
prob = ODEProblem(rhs, u, (0f0, dt), ( $\theta$ , d))  
integ = init(prob, SimpleRK4(); dt = dt_sub)  
@trace mincut=true for _ in 1:n_substeps  
    step!(integ)  
end
```

A genuine drop-in: same loss as the hand-written RK4 to five digits, no runtime penalty.

Adaptive solvers. For adaptive solvers this does not work: they pick the step count and dt at run time, from the error estimate — so when Reactant traces the program, there is no loop of known length to wrap the annotation around.

Experimental: DiffEqReactant.jl.

- A Tsit5 that is traceable by Reactant — adaptivity expressed so the compiler can see it.
- Already trains a UDE with Lux + Reactant + Enzyme on the GPU.
- Very experimental; a deeper SciML integration is the goal.

github.com/SebastianM-C/DiffEqReactant.jl

Fixed-step SciML solvers drop in today; traceable adaptive solvers are the next step.



`github.com/arno-papers/CDC2026`

Models, training, evaluation and every figure in this talk.

Paper: `arXiv:2603.16146`

The resources and services used in this work were provided by the VSC (Flemish Supercomputer Center), funded by the Research Foundation – Flanders (FWO) and the Flemish Government.